Name - Prasad Sanjay Vangujare

Class - SY.B [AI & DS]

Roll No. - B55

Subject - PDS (Ex.10)

Aim - To study and understand Version Control using Git and GitHub, and perform basic operations like adding, committing, and pushing files to a repository.

Theory - Version Control using Git & GitHub

Version Control Systems (VCS) are software tools that help software teams manage changes to source code over time. They keep track of every modification to the code in a special kind of database. If a mistake is made, developers can turn back the clock and compare earlier versions of the code to help fix the mistake while minimizing disruption to all team members.

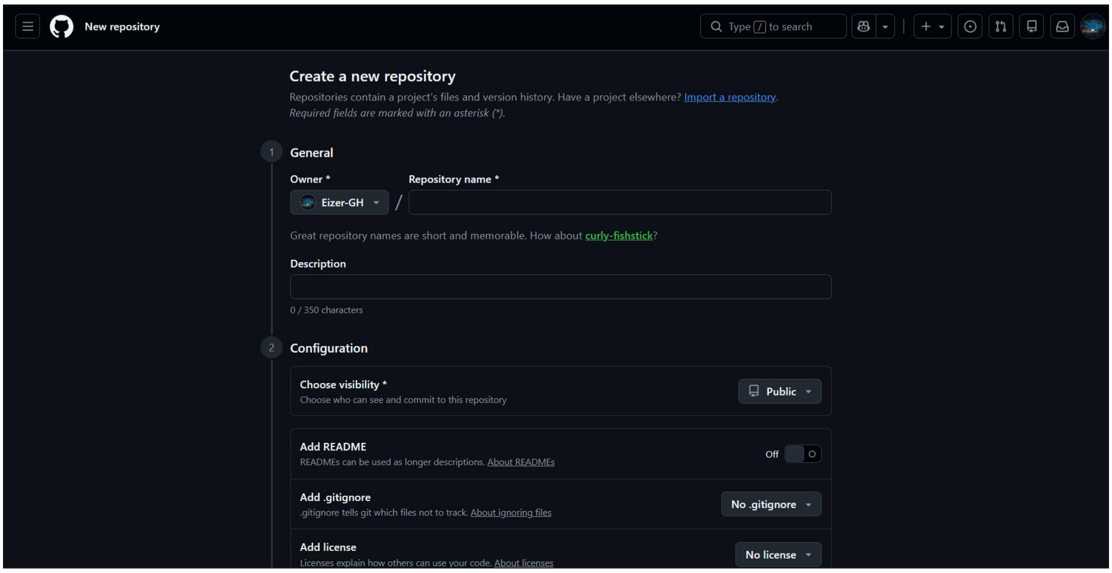
Git is a Distributed Version Control System (DVCS). In a DVCS, every developer has a full copy of the project history on their local machine. This makes it fast and allows for offline work.

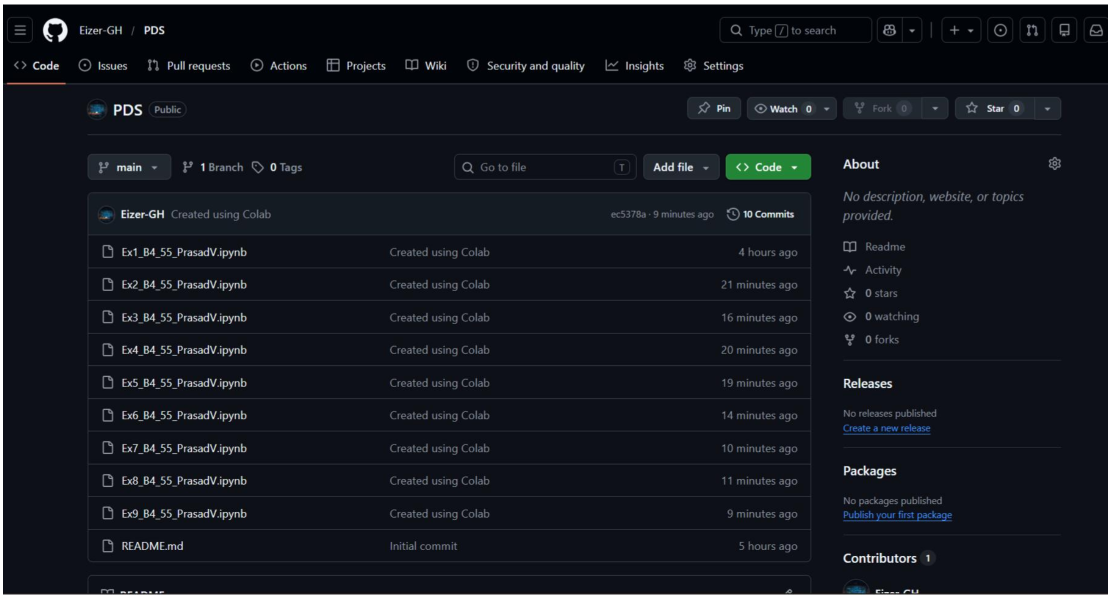
GitHub is a cloud-based hosting service that lets you manage Git repositories. It provides a web-based graphical interface and adds collaboration features like bug tracking, task management, and "Pull Requests" for team reviews.

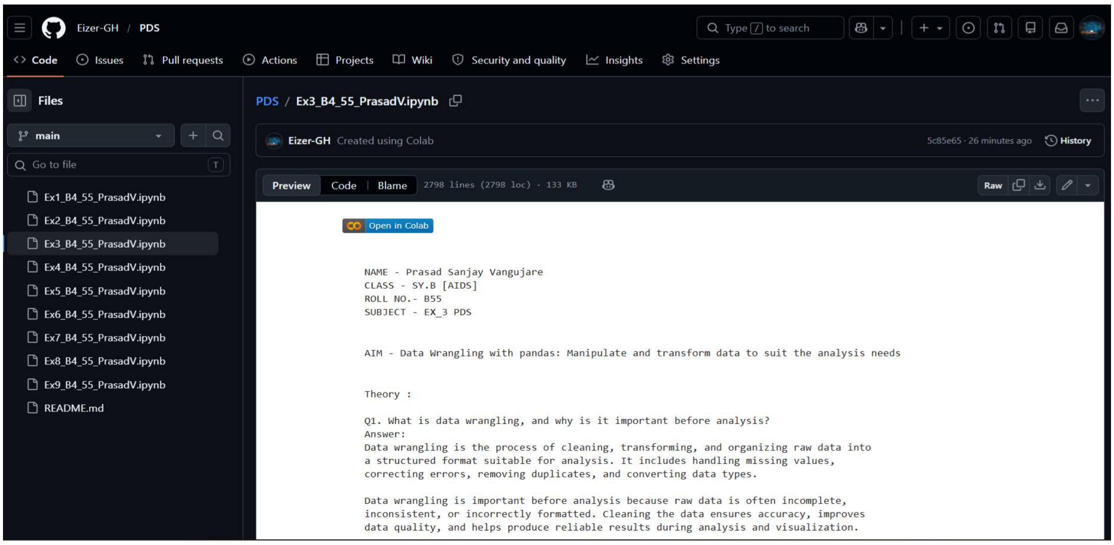
Basic workflow in Git:

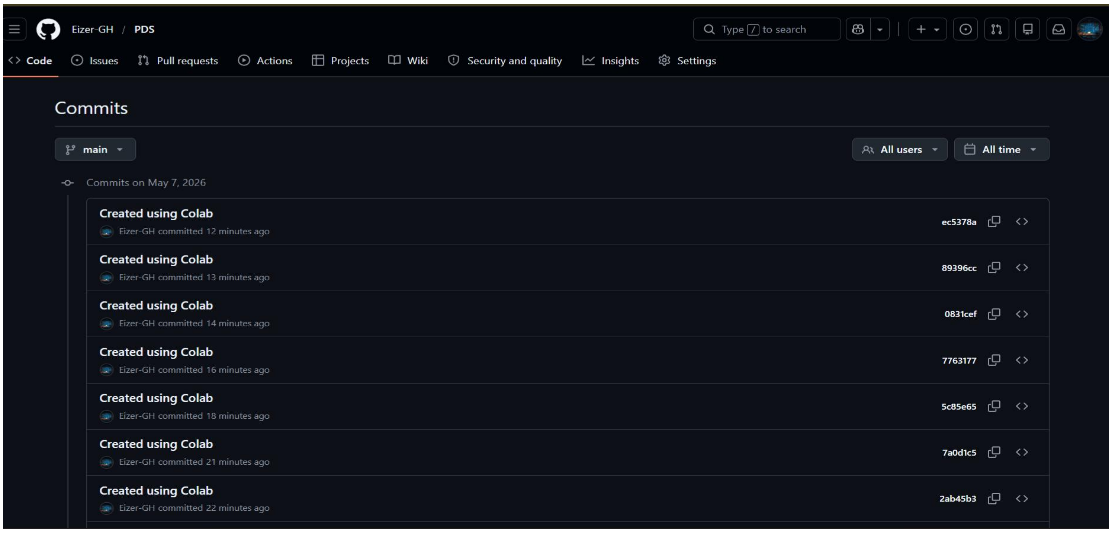
- Create repository
- Add files
- Stage files (git add)
- Commit changes (git commit)
- Push to remote repository (git push)

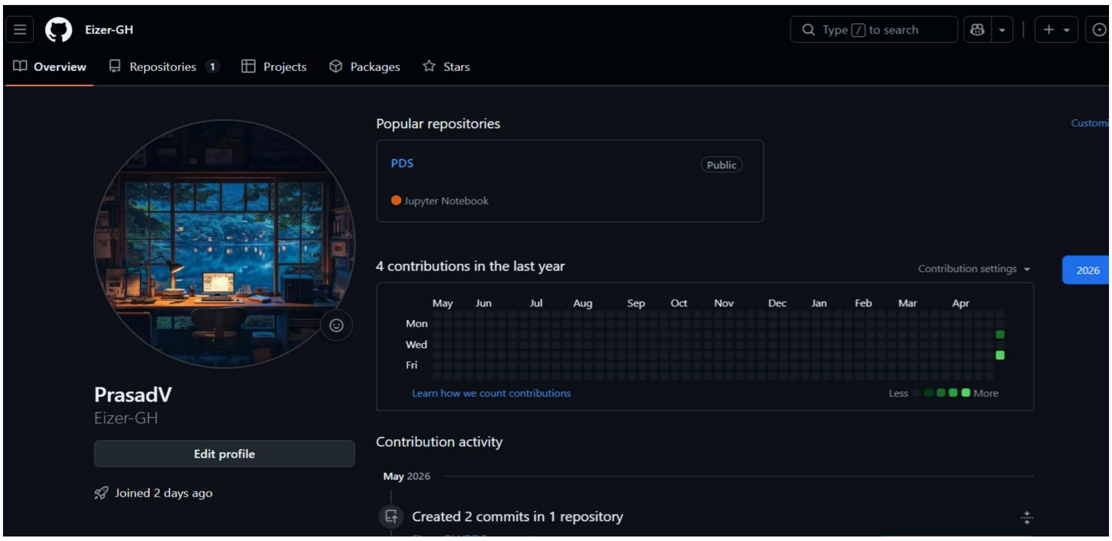
Screenshots -






Eizer-GH / PDS
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
main
Go to file
Ex1_B4_55_PrasadV.ipynb
Ex2_B4_55_PrasadV.ipynb
Ex3_B4_55_PrasadV.ipynb
Ex4_B4_55_PrasadV.ipynb
Ex5_B4_55_PrasadV.ipynb
Ex6_B4_55_PrasadV.ipynb
Ex7_B4_55_PrasadV.ipynb
Ex8_B4_55_PrasadV.ipynb
Ex9_B4_55_PrasadV.ipynb
README.md
PDS / Ex3_B4_55_PrasadV.ipynb
Eizer-GH Created using Colab
5c85e65 · 26 minutes ago
History
Preview
Code
Blame
2798 lines (2798 loc) · 133 KB
Raw
Open in Colab
NAME - Prasad Sanjay Vangujare
CLASS - SY.B [AIDS]
ROLL NO.- B55
SUBJECT - EX_3 PDS
AIM - Data Wrangling with pandas: Manipulate and transform data to suit the analysis needs
Theory :
Q1. What is data wrangling, and why is it important before analysis?
Answer:
Data wrangling is the process of cleaning, transforming, and organizing raw data into
a structured format suitable for analysis. It includes handling missing values,
correcting errors, removing duplicates, and converting data types.
Data wrangling is important before analysis because raw data is often incomplete,
inconsistent, or incorrectly formatted. Cleaning the data ensures accuracy, improves
data quality, and helps produce reliable results during analysis and visualization.


Eizer-GH / PDS
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Commits
main
All users
All time
Commits on May 7, 2026
Created using Colab
Eizer-GH committed 12 minutes ago
ec5378a
Created using Colab
Eizer-GH committed 13 minutes ago
89396cc
Created using Colab
Eizer-GH committed 14 minutes ago
0831cef
Created using Colab
Eizer-GH committed 16 minutes ago
7763177
Created using Colab
Eizer-GH committed 18 minutes ago
5c85e65
Created using Colab
Eizer-GH committed 21 minutes ago
7a0d1c5
Created using Colab
Eizer-GH committed 22 minutes ago
2ab45b3


Eizer-GH
Type / to search
Overview
Repositories 1
Projects
Packages
Stars
Popular repositories
PDS
Public
Jupyter Notebook
4 contributions in the last year
Contribution settings
2026
May Jun Jul Aug Sep Oct Nov Dec Jan Feb Mar Apr
Mon
Wed
Fri
Learn how we count contributions
Less
More
PrasadV
Eizer-GH
Edit profile
Joined 2 days ago
Contribution activity
May 2026
Created 2 commits in 1 repository

Questions -

1. What is the difference between Git and GitHub?

- **Git :** It is a local software/tool installed on your computer. It is responsible for tracking changes in your files and managing the version history of your project.

- **GitHub :** It is a web-based platform that hosts Git repositories in the cloud. It allows multiple people to collaborate on the same project from different location.

2. Explain the difference between git add and git commit

- **git add :** This command moves changes from the working directory to the Staging Area. It tells Git which specific file modifications you want to include in the next "snapshot."

- **git commit :** This command takes all the changes currently in the Staging Area and saves them permanently to the **Local Repository**. Each commit acts as a checkpoint with a unique ID and a descriptive message.

3. What is the staging area in Git?

The staging area (also known as the "index") is an intermediate area between your working directory and the local repository. It allows you to format and review your changes before committing them. You can pick and choose which files to stage, ensuring that each commit is clean and logical.

4. What happens if you modify a file after committing it?

If you modify a file after a commit, the file's status changes back to "Modified" in your working directory. The version in your local repository remains unchanged until you perform a new git add (to restage the new changes) followed by another git commit.

5. Difference between git push and git pull

git push : This command uploads your local repository commits to a remote repository (like GitHub). It "pushes" your changes so others can see them.

git pull : This command fetches changes from the remote repository and immediately merges them into your current local branch. It is used to keep your local work synchronized with the team's latest updates.

6. What is Version Control? Differentiate between Centralized vs. Distributed VCS.

Version Control is the practice of tracking and managing changes to software code. It provides a history of development and enables collaboration.

Feature	Centralized VCS (CVCS)	Distributed VCS (DVCS)
Storage	One single central server contains the full history.	Every user has a full copy of the entire repository history locally.
Connectivity	Requires a network connection to perform most operations (commit, view history).	Most operations (commit, log, diff) are performed locally without internet.
Risk	If the central server fails, the project history can be lost if not backed up.	Since every user has a copy, any local repository can act as a backup.
Examples	SVN (Subversion), Perforce.	Git, Mercurial.